

Technical Director: Mark Boutwell - maboutw@radium.ncsc.mil



Brian Ricks - bwr031000@utdallas.edu

Table of Contents

1 Project Summary	2
1.1 Project Description	2
1.2 Specific Aims	2
1.3 Motivation	2
2 Executive Summary	2
3 Research Project Plan and Approach	3
4 Data Masking Techniques	3
4.1 Encryption	4
4.1.1 Encryption Threats	4
4.2.2 Encryption Mitigations	5
4.2 Substitution	5
4.2.1 Substitution Threats	6
4.2.2 Substitution Mitigations	6
4.3 Shuffling	6
4.3.1 Shuffling Threats	7
4.3.2 Shuffling Mitigations	7
4.4 Nulling Out and Masking Out	7
4.4.1 Nulling Out	7
4.4.2 Masking Out	7
4.4.3 Nulling and Masking Out Threats	8
4.4.4 Nulling and Masking Out Mitigations	8
5 Results	8
6 Applications of Data Masking for Cybersecurity Research in Active Defense	10
6.1 Policy-Based Data Masking	10
6.2 Source Code Obfuscation	10
6.3 Command Obfuscation	11
6.4 Security in Cloud Computing	11
7 Future Work	13
8 Conclusion	13
9 Investigator Biographies	13
10 Bibliography	15

1 Project Summary

1.1 Project Description

With privacy preservation becoming an increasingly challenging problem, many models of masking data have been introduced. This has been accompanied by some computation approaches each having their own efficiency factors. The main goal of this project is to identify approaches and designs for masking data to preserve privacy.

1.2 Specific Aims

Since the subject matter is relatively unexplored and highly advanced, we will be creating a survey paper which highlights the theories of data masking. Through this research project, our team will introduce existing data masking models and do an in-depth analysis of each model which would cover their differences, similarities, effectiveness, and efficiency. This project would assist security engineers by consolidating the details in a single paper which would further help them apply the most optimized data masking model to protect the privacy of users based on the requirements of the system.

1.3 Motivation

In today's information age, the data is an important asset of the organization so the security of the data is a vital role in the industry. In order to achieve the above aspect, the data masking is used. The data masking is mainly the data that is replaced with realistic but not the original data. The main objective is to make sensitive information not made available outside of the environment. The data masking is to just provide the copy of the production data in support of the development environment and thus it controls the leakage of the data.

There are many different approaches to data masking. How to choose them in some specific environment is a question for the reseathers. Also, there are many issues associated with masking data. How to improve the privacy preservation across systems is also needed to be investigated. These questions above motivate us to do research in data masking.

2 Executive Summary

With the rapid development of digital technology and the widespread popularization of the mobile Internet, more and more people's daily life and work information are digitized and collected, and used by massive mobile terminals and intelligent processing equipment, resulting in the explosive growth of the amount of data on the Internet and contributed to the arrival of the era of big data. Big data contains huge commercial value. At present, all walks of life are committed to the mining and analysis of big data. However, there are also a series of privacy issues that follow. Big data often contains a large amount of sensitive information of users. The

direct release or sharing of original data without processing will inevitably lead to leakage of user privacy. In recent years, the issue of data privacy protection in the computing environment has received extensive attention. The data privacy protection models and technologies have been developed to some extent.

There are many approaches to protect the privacy of the users. However, with the increasing volume of data, more diverse data types, and increasingly complex storage systems, this means that data privacy protection models, algorithms, and related technologies are facing many problems. This includes issues associated with properly masking the data and processing the masked data to minimize the potential of leaking information. Even when masked, data can be vulnerable as it is sent and received across system and architectural boundaries.

The main goal of this project is to identify approaches and designs for masking data to preserve privacy. This project will be divided into four phases, consisting of rigorous research.

3 Research Project Plan and Approach

Our Research project was completed in three phases. We started by studying the characteristics of common masked data, the existing approaches to preserve their privacy and common attacks against this data, where we look at the issues associated with data masking and the implications of using different techniques for computing architecture. We then created a threat model and attack vectors where we researched each technique's weakness that an attacker could exploit. We also suggested mitigation to avoid these attacks. After this, we will investigate implications of using these techniques with different architectures where we research what are the necessary features to enhance privacy preservation across system and architectural boundaries.

4 Data Masking Techniques

Data masking refers to hiding sensitive data in a database from unauthorized users. In data masking, the information is not necessarily completely hidden or removed. Instead, the data is modified so that less secret information is visible in order to protect user privacy or conform to privacy policies. The resulting data after masking resembles the real, original data and can be used for computing purposes without sharing secret information. One of the primary problems in data masking is balancing disclosure risk with data utility.

Disclosure risk refers to the likelihood of a malicious viewer being able to infer who a certain record belongs to, or the true value of a sensitive field. Data utility refers to the usefulness of the data for computing purposes. Low data utility means that analysis of the masked data may yield very different conclusions and statistical inferences than that of the original unmasked data. This trade-off of disclosure risk and data utility creates a large set of data masking techniques.

Data masking techniques can be put into two categories: static and dynamic. Static data masking, or SDM, alters data at rest. It permanently changes the original database so that there is no unmasked sensitive data available. Using SDM means that there is no additional processing cost when performing queries as the data is already masked. SDM is often utilized for complying with privacy regulations, since it ensures that a database does not have any copies of unmasked data. Dynamic data masking, or DDM, alters data in transit. The data is masked after retrieval from the database, leaving the original database intact and masking the query results. This allows authorized users to view sensitive data, while providing masked data to unauthorized users. DDM can also be used to mimic role-based access by applying different levels or types of masking depending on the user making the request.

4.1 Encryption

Encryption is the most common and one of the oldest algorithm approaches for information security. Encryption is the encoding of text using private key algorithms. Although encrypted data appears random, encryption proceeds in a logical way, allowing a party that receives the encrypted data and possesses the right key to decrypt the data, turning it back into the original data. Highly secure encryption will use keys complex enough that an attacker is unlikely to decrypt or break the ciphertext by plain brute force, that is by guessing the key.

This means that encryption has a high cost to attack, even when the attacker knows what algorithm is used, while maintaining a very low disclosure risk. Compared to other masking methods, encryption requires greater computation overhead and hence is costlier to implement. It is also excessive in most cases, as most fields do not require this level of security. Encrypting the entire base also eliminates data utility, as it cannot be useful without decryption, unless homomorphic encryption is used (which allows computation on encrypted data without decrypting it first).

4.1.1 Encryption Threats

Searchable encryption (SE) is a cryptographic technique for maintaining data privacy and security. The data owners in SE can send the encrypted information onto the cloud service platform that can perform search operations over the encrypted data without decrypting it. The encrypted data includes encrypted indexes and encrypted documents. The data users can send encrypted queries to retrieve the relevant information. To enable search, the data owners need to outsource the searchable encrypted indexes onto the cloud servers. The indexes are generated from the keywords in the encrypted documents. Therefore, there are some attacks that exist in SE due to the vulnerabilities in SE schemes. For instance, the cloud provider can utilize the encrypted ranking information to infer the keyword information in the documents. The leakage of search patterns can be used to infer the information of queries and then the keywords of these

queries. Keyword guessing attack can guess some candidate keywords and encrypt these keywords by comparing with actual encrypted keywords.

The attacker, armed with the encrypted text and encryption algorithm used, can perform exhaustive (brute force) search on the key space to determine the key and decrypt the encrypted text to access the plaintext sensitive information. Then they can produce candidate plaintexts and observe if they make sense. Brute forcing encryption does not require much skill, but it needs a powerful enough computer with sufficient CPU, RAM and HD. A supercomputer would only take 4 seconds to brute force a key of 32 characters. Doubling the key length to 64, it would take the same computer about 585 years to generate all possible keys. This does not mean that a cipher of a 64 bit key can never be broken by brute-force. Although the time taken to search through every key is very big and it would not be possible to check all keys, there is a possibility that the correct key could be found in a short time (example, the key can be found in a day) into the search process. Once a correct key is found, we no longer need to search through the other keys.

4.2.2 Encryption Mitigations

Using commonly accepted encryption algorithms and recommended key sizes can mitigate this threat. Not only should the key be very long but also be randomly generated since Brute force time would be significantly reduced, if a weak key is used and a hybrid dictionary attack is accompanied. Furthermore, the honey encryption technique has been applied as a good strategy to combat the brute-force attack. The main idea of honey encryption is basically to confuse the attacker, so that it gets frustrated while using brute-force to decipher the confidential information. In ordinary encryption techniques like AES, when the attacker uses incorrect keys to decrypt the plaintext information, the message obtained will not make sense, but honey encryption can produce a plausible outcome which means the message is fake but not senseless. In this case, the attacker will believe the message obtained with any combination of keys is the original message, then the search space will never be exhausted. The format-preserving encryption (FPE) technique is one of terms related to honey encryption. FPE tries to maintain the format of ciphertext same as the original text, so that we can lure the attacker.

4.2 Substitution

Substitution is the technique of replacing the existing data in the database with some random values from a pre-prepared dataset that look real and are of the same type as the original data. For example, one rule could be that all first names starting with A are replaced by Aaron. The modified data may or may not be related to the original data. Doing so verifies that we achieve low disclosure risk since there could be multiple similar entries in the database. Another advantage of this technique is that the look of the data remains intact even after substitution.

The downside of the method is that the random list must be available for each column to be replaced. Thus, the system should have the ability to generate invalid but similar data with the same data type. It also means that we get a low data utility.

4.2.1 Substitution Threats

In the example, replacing every name that starts with A with Aaron, an attacker may be able to guess at the substitution rule. To not reveal information about the original data, use substitutions that are not related to the original data so that it will be difficult for a malicious actor to determine the substitution rules.

4.2.2 Substitution Mitigations

This can also be avoided by having multiple different possible substitutions available, so it is not a one-to-one mapping. This mitigation can also make it less obvious that the data is masked at all, or that substitution was applied to the records, further frustrating an attacker's attempts to gain useful information about their targeted individual or individuals.

4.3 Shuffling

Shuffling refers to swapping data fields across different records without modifying the values of the fields themselves. The shuffling can be done on individual fields or on groups of columns to retain data consistency. For data with relationships across fields, they may be grouped to prevent shuffling from creating incorrect-looking data; for example when separate columns include a phone number and state, so that the shuffled data still remains plausible. If shuffling columns as a group is still a privacy concern, partitions can be made within groups so additional shuffling can be done without creating inconsistent data. Following the previous example, we could create a partition for each state and shuffle phone numbers across entries within the same state.

This process works best for when metrics or operations are being computed on the entire dataset, as all statistical values would remain the same. Unfortunately most shuffling methods have a high disclosure risk, as all the original data is still in the database. As such, some sensitive information can be inferred by malicious users. For example, if every record includes a medical condition, it can be inferred that any individual user has one of the conditions in the database. Shuffling fields may also lead to low data utility if it is not done with careful grouping or when not computing on the whole dataset, as the analysis of the shuffled data may yield very different statistical values. These issues can limit the efficacy of shuffling for masking data for computing purposes.

4.3.1 Shuffling Threats

An attacker who can view entries may be able to surmise the presence of a certain individual or individuals based on outside known information about their target. The existence of certain types of records (for example, a very high value or a location that may be unique to certain individuals) may betray the existence of specific records in the data. An attacker who can perform statistical analyses on the data is able to draw conclusions about the group as a whole that can be applied to individuals as well, even when the columns are shuffled.

4.3.2 Shuffling Mitigations

To mitigate this we can perform global sensitivity analysis. This can be done on unmasked data as well for privacy, but is useful for shuffling as it will affect statistical analysis. This asks, what is the sum of the worst case difference in answers that can be caused by adding or removing an individual from the dataset? For example, the average age in your dataset could dramatically increase or decrease if you add a very old or very young person; would this give information away? Think of questions that an attacker might want from your dataset.

To reduce the global sensitivity gap, we can add Laplacian noise to the data. This is done by using random values from a Laplacian distribution that has a standard deviation large enough to span and cover the gap. This way, the distribution of possible results from the data coincides with their neighbor's distribution of results. This reduces data utility slightly, but you can still know the general area of the original values.

4.4 Nulling Out and Masking Out

4.4.1 Nulling Out

The nulling out technique just deletes sensitive data and replaces the field with NULL values. For example, we could replace a phone number with —(XXX)XXX-XXXX, or a Social Security Number (SSN) with XXX-XXXX-XX. It is a simple way of ensuring that the data is completely invisible, hence ensuring that there is no disclosure risk. However, this also means that there is no data utility for the nulled out fields. For instance, it is difficult to write and test customer account maintenance forms if the customer name address and contact details are NULL values. Therefore, nulling out is only useful in circumstances where the data is not required, or for extremely sensitive values, but is rarely useful as the entire data sanitization strategy.

4.4.2 Masking Out

Character scrambling or masking out of certain fields is also another simplistic yet very effective method of preventing sensitive information to be viewed. It is really an extension of nulling out, but there is a greater emphasis on keeping the data real and not fully masked all together. Masking data, besides being the generic term for the process of data anonymization, means

replacing certain fields with a mask character (such as an X). This effectively disguises the data content while preserving the same omission on front end screens and reports. For example, an ecommerce call center operator might only see the last 4 digits of a customer's credit card but once the billing system passes the customer's details for charging, the full number is revealed to the payment gateway systems.

If the data is in a specific, invariable format, then masking out is a powerful and fast option. If numerous special cases must be dealt with then masking can be slow, extremely complex to administer and can potentially leave some data items inappropriately masked. Furthermore, it should be ensured that neither too little nor too much of the data is masked. For example, if everything except the last two digits of the credit card, that would be too insecure as those digits could be easily generated by the checksum card number generation algorithm. On the other hand, if just the last two digits are masked, that would mask away important information about the card that could be needed.

4.4.3 Nulling and Masking Out Threats

Though nulling or masking out is typically used on the unique identifiers in a dataset, the subsequent columns that are left unchanged are at risk of being quasi-identifiers (QIs) that resourceful attackers can still use to identify individuals. An attacker armed with publicly available information or information gathered from any other sources may be able to cross-reference the QIs to identify individuals from seemingly innocuous data.

For example, birth data can be inferred from several offline and online sources, including data brokers, voter registration lists, online white pages, or social media profiles. With complete availability of birth data for individuals born between 1989 and 2003, the first 5 digits of 26 million SSNs may be correctly matched at the first attempt and almost 5 million complete SSNs may be matched with <1,000 attempts.

4.4.4 Nulling and Masking Out Mitigations

k-anonymity is a property of a dataset where each record should be indistinguishable from at minimum k-1 records based on the data's QIs. As a result, the cardinality of any query to the dataset should be at least k. To achieve k-anonymity, the remaining unmasked columns must be partially or completely generalized (nulled out) until there are enough indistinguishable records.

5 Results

5.1 What is the most efficient AND effective data masking solution?

Each data masking has its own advantages and disadvantages as we can see from this table of data masking techniques and its properties. The efficiency and effectiveness of a technique is based on the data and its uses.

	Encryption	Shuffling	Substitution	Nulling Out	Masking out
Data type	No	Yes	Yes	No	Partially
Aggregate value	No	Yes	No	No	No*
Frequency distribution	Yes	Yes	No	No*	No*
Uniqueness	Yes	Yes	No	No	No*
Data Disclosure Risk	Low	High	Low	None	High
Data Utility	Low	Low	Low	None	High

5.2 Considering known risks and challenges (e.g., data leakage, ease of technique identification and defeat) does computing on masked data overcome the risk of leaks due to unmasking?

This investigation has revealed that simply masking the data is often not enough to protect user privacy, however it can help mitigate data leaks. There are many additional considerations that must be taken into account that are dependent on the masking technique used, as different techniques have different benefits and pitfalls. The ideal mitigation strategy for a given dataset is also dependent on the types of data stored and how the data is used, as increased privacy often leads to lower data utility. When masking the data and applying additional threat mitigations, it would be beneficial to consider what information malicious actors would want from your dataset and what their motivations may be.

5.3 Is a hybrid data masking technique feasible?

Yes! In fact, utilizing multiple different masking techniques in tandem can further protect privacy, especially as it may not be clear to an attacker what techniques have been applied to the dataset. Strategically-chosen technique combinations can frustrate malicious attempts to gain information about individuals or the group as a whole. Adding additional mitigations for those techniques provides even more protection. For example, combining shuffling & masking out, and enforcing a high k-anonymity would make it very difficult to infer data about individuals.

5.4 Is a hybrid masking architecture feasible? Mask only specifically tagged data fields instead of the whole of the dataset.

No. If the unmasked fields are quasi identifiers, data leaks can still happen due to circumstance and information that can be inferred.

6 Applications of Data Masking for Cybersecurity Research in Active Defense

6.1 Policy-Based Data Masking

Customer data is protected by data privacy laws or policies, which indicates which kind of data is sensitive, and how to protect the data. Masking is often done in order to conform to such regulations and must also be taken into consideration when identifying a technique for the situation. There are the four most common policies. California Consumer Privacy Act (CCPA) is a law that entitles Californians to know how their private data is collected, used, shared, or sold, and to withdraw their data content for future use. Customers also have the right to sue companies if the company doesn't follow security policy to protect their personal data to make them become victims of a data breach. Healthcare Insurance Portability and Accountability Act (HIPAA) is a data privacy law that protects patients' personal information against reasonably anticipated threats, such as unauthorized use or disclosure of their private data, or hazards to the integrity of protected health information. The Gramm-Leach-Bliley Act (GLB Act or GLBA) contains rules on how financial institutes collect, protect, use, and share their customers' personal and financial information, which covers data stored in physical documents and electronic or other mediums. The General Data Protection Regulation (GDPR) which states that companies must have appropriate technical and organizational measures to protect Europeans' personal data against being stolen, altered, destroyed, lost or disclosed to or accessed by unauthorized individuals while it is transmitted, stored or processed.

6.2 Source Code Obfuscation

Among the various techniques available for protecting code from different Man-At-The-End attacks, source code obfuscation is one of the most popular choices, deployed to prevent code comprehension. Source code obfuscation consists of code transformations that make a program more difficult to understand by changing its structure, while preserving the original functionalities.

There are 3 main types of source code obfuscation techniques. Layout obfuscations remove relevant information (such as identifier names) from the code without changing its behavior. Control-flow obfuscations alter the original flow of the application. Data obfuscations transform application data and data structures (e.g., data encoding, data splitting).

A determined attacker can still spend enough time to inspect obfuscated code, and might locate the functionality to alter and succeed in their malicious purpose. Therefore, obfuscation is rarely deployed alone and is complemented by other approaches.

6.3 Command Obfuscation

SQL command obfuscation is a common attack to bypass web application firewalls, execute SQL injection attacks and implicitly run SQL commands against databases. One of the most common methods to perform a SQL injection attack is using SQL HEX encoding, where plain text readable SQL transactions are converted to HEX characters, hence making them unreadable and more difficult to monitor.

Most instances of SQL injection can be prevented by pre-processing the sql input like changing the value back to base string along with using parameterized queries (also known as prepared statements) instead of string concatenation within the query.

6.4 Security in Cloud Computing

Cloud computing is a very vast and rapidly emerging technology. Adversaries may collect sensitive data from these cloud storage solutions. The cloud service providers typically offer security guides to help end users configure systems, though misconfigurations are a common problem. Furthermore, many cloud providers provide Machine Learning as a Service (MLaaS) as computing platforms and learning frameworks for machine learning model training and serving. Therefore, the use of cloud-based inference models not only requires accurate predictions but also needs to prevent the information leakage of intellectual property since the training model involves medical records, financial data, business, or other types of sensitive information.

There are many kinds of possible threats when the cloud provider or end user are malicious. They can retrieve critical information directly from the training dataset, model parameters, or model output when deploying machine learning applications in the cloud. The cloud backend may follow the privacy preservation mechanism and honestly serve the user end, but the backend may misuse the extracted private information for unauthorized purposes, e.g., targeted advertising and political advocacy, which are not relevant to the objective of the machine learning application. The edge devices may collude with the backend in finding out other edge devices' privacy contained in the data. The colluding participants will faithfully transmit their

inference data with or without obfuscation, but the privacy protection for an edge device needs to be maintained when edge devices are colluding with the cloud backend.

The popular solution is to process the training data using the encryption technique before releasing them to untrusted third parties. The homomorphic encryption (HE) has been prominently introduced to process the training datasets. HE is a kind of public-key encryption. It enables a server to perform computation directly on the data while it is still encrypted, and the results of the computations remain in encrypted form.

The homomorphic encryption has been proposed to combine with other techniques as well. For instance, a new data masking technique has been proved as an efficient privacy-preserved protocol by perturbing data through multiplying a rational number with encrypted data. The encryption and obfuscation alone are not sufficient for information security in cloud storage because the attackers can get information by attack like brute force or reverse engineering. Therefore, the encryption requires integration with obfuscation techniques. Data can be encrypted before transmitted to the cloud provider based on sensitivity of data and user kept key secret gives security to data in transition. The obfuscation technique can be applied to the service provider for security.

Specifically, the encryption ensures that the raw data remains confidential since the cloud does not have access to the keys needed to decrypt it. However, the privacy of inference data or prediction are still not preserved. CryptoNets has been proposed to solve this problem. The main ingredients of CryptoNets are homomorphic encryption and neural networks. It is a method to convert learned neural networks to CryptoNets that can be applied to encrypted data to make encrypted predictions and also return them in encrypted form. The owner who has the secret key can decrypt encrypted output. Therefore, both accuracy and security are achieved by combining homomorphic encryption with privacy-preserving machine learning approaches.

Because of the significant compute overhead to the edge devices from the CryptoNets, a lightweight obfuscation neural network (ObfNet) approach has been proposed to obfuscate the inference data before transmission to the cloud backend. The key idea of ObfNet is that it is a small-scale trained neural network. The ObfNet uses a many-to-one mapping activation function, so the backend cannot estimate the exact original data since there exist infinite possible inputs mapping to the same output.

While homomorphic encryption can prevent data leakage, it is not generic for arbitrary machine learning algorithms and the performance is not very efficient. An obfuscation function applying to the training data before feeding to the backend has been proposed for privacy-preserving machine learning. This function adds random noise to the existing training dataset to avoid the attacks that attempt to steal the information from the individual samples. For the attacks to the groups of samples, this function augments the dataset with adding new noisy samples. The injected noise only obfuscates the attacker's observation but does not affect the prediction accuracy of the training models. In this case, the customers can apply this function to the

sensitive training datasets and disclose to the cloud service provider without worrying about the machine learning privacy attacks.

7 Future Work

Based on the research done these past few months, our team has suggested additional areas of research which would highly benefit the topic at large. This future work could be divided into two sections: data masking techniques and applications of data masking.

For data masking techniques, in the future, we would like to create working prototypes and penetration testing attacks to codify our conclusion. We would also want to look at if and how semantic and referential integrity is preserved when there are multiple masked tables in a database. Another topic of research could be dynamic data masking and its applications. Dynamic data masking allows surgical masking of specific sensitive data while leaving the remaining non-sensitive data in its original form.

There are many applications for data masking and we would like to look into some of those. Firstly, the use of Dynamic Binary Instrumentation to inject code that performs targeted identification, tracing, functional classification, and/or bypass of a respective data masking technique. We could also look at the exploration of data masking techniques in a Zero-Trust Framework.

8 Conclusion

We would like to conclude by reiterating that there is no one size fits all for data masking techniques and algorithms must be selected on a case-by-case basis that takes many factors into account, including how the data is used and possible attackers' resources and motivations. Although the research topic is fairly new, there is a huge scope of further research on the masking techniques as well as applications.

9 Investigator Biographies

Megan Tran is currently pursuing a masters in Computer Science with the Cybersecurity track at the University of Texas of Dallas. She has worked on web and Java development for over two years at Sandia National Laboratories. She has also interned at the Air Force Research Laboratories working in embedded systems security. At UTD, she has completed coursework in information security, network security, data & application security, language-based security, and operating systems. She is a Women in Cybersecurity scholarship recipient and has consistently placed in the top 10% of participants at National Cyber League competitions.

Saloni Shivdasani is a masters student at the University of Texas at Dallas majoring in Computer Science with a concentration in cyber security. She has been in the software development

industry for more than 2 years. At her current job at a consulting firm, she has been utilizing skills in project management, cloud administration, continuous integration and delivery on the cloud and full stack mobile development. She also has had a year of experience as an Engineering Analyst at the Information Security Office at UTD utilizing cloud administration along with Python and shell automation. While at UTD, she had taken classes pertaining to information security, data and application security, network security, and language based security along with database systems, telecommunication systems, advanced operating systems and software testing. Outside of university and work, she has taken part in security competitions where she has consistently placed in among the top 10% nationally and received scholarships to attend multiple conferences. She is currently studying to get her security+ credentials.

Yingyuan Jiang is a master student with a GPA of 3.81/4.0 at UTD majoring in Computer Science with a track in Data Science. He has four years of industrial and academic experience as a software developer on cloud platforms with 3 published papers about security and 12 patents. In the previous experience, he designed the database and website on Alibaba Cloud(the biggest cloud platform in China) for a technology company. He was selected from thousands of candidates to be voluntarily a student expert in Alibaba Security to make contributions to projects related to security risk because he got 2nd place of 2520 teams in IJCAI-2019 Alibaba Adversarial AI Challenge (AAAC 2019), an international attack-and-defense competition, where he applied encode-and-decode architecture to train models using masking image data to defend against the attack. In 22' Summer, he did an internship in a local company in Dallas, Texas and worked on the projects about database design and backup related to customers' private data. Also, he has taken courses, such as Database Design, Big Data Management and Analytics, Web Programming. He is comfortable with Java and Python.

Hua Xiao is currently a master student with GPA of 4.0/4.0 in Computer Science at the University of Texas at Dallas on the Intelligent Systems Track. She has received a PhD degree from Virginia Tech and has 8 years of industrial and academic experience in the Biotechnology area. Her background experience is related to the biosecurity and biosafety based on modern biotechnics in agriculture and food production with 3 published papers in the peer-reviewed journals. She is now doing an internship in a local company in Plano, Texas and working on the projects about web application development and QA testing. She has taken coursework for Information Security, Machine Learning, Database Design, Algorithm Design, Operating Systems, Big Data Management and Analysis and Computer Architecture. She has a strong background in Python and some experience in Java, MIPS, HTML/CSS, JavaScript and PHP.

10 Bibliography

- [1] T. Zhang, Z. He, and R. B. Lee, "Privacy-preserving Machine Learning through Data Obfuscation." arXiv, Jul. 12, 2018. Accessed: Nov. 04, 2022. [Online]. Available: <http://arxiv.org/abs/1807.01860>
- [2] G. Qiu, X. Gui, and Y. Zhao, "Privacy-Preserving Linear Regression on Distributed Data by Homomorphic Encryption and Data Masking," *IEEE Access*, vol. 8, pp. 107601–107613, 2020, doi: [10.1109/ACCESS.2020.3000764](https://doi.org/10.1109/ACCESS.2020.3000764).
- [3] D. Xu, M. Zheng, L. Jiang, C. Gu, R. Tan, and P. Cheng, "Lightweight and Unobtrusive Data Obfuscation at IoT Edge for Remote Inference," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9540–9551, Oct. 2020, doi: [10.1109/JIOT.2020.2983278](https://doi.org/10.1109/JIOT.2020.2983278).
- [4] N. Dowlin, R. Gilad-Bachrach, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," p. 10.
- [5] P. Bajpai and R. Enbody, "Attacking Key Management in Ransomware," *IT Prof.*, vol. 22, no. 2, pp. 21–27, Mar. 2020, doi: [10.1109/MITP.2020.2977285](https://doi.org/10.1109/MITP.2020.2977285).
- [6] D. V. N. Siva Kumar and P. Santhi Thilagam, "Searchable encryption approaches: attacks and challenges," *Knowl Inf Syst*, vol. 61, no. 3, pp. 1179–1207, Dec. 2019, doi: [10.1007/s10115-018-1309-4](https://doi.org/10.1007/s10115-018-1309-4).
- [7] A. K. Agarwal, L. Rani, R. G. Tiwari, T. Sharma, and P. K. Sarangi, "Honey Encryption: Fortification Beyond the Brute-Force Impediment," in *Advances in Mechanical Engineering*, G. Manik, S. Kalia, S. K. Sahoo, T. K. Sharma, and O. P. Verma, Eds. Singapore: Springer Singapore, 2021, pp. 673–681. doi: [10.1007/978-981-16-0942-8_64](https://doi.org/10.1007/978-981-16-0942-8_64).
- [8] K. Suthar and J. Patel, "EncryScation: An Secure Approach for Data Security Using Encryption and Obfuscation Techniques for IaaS and DaaS Services in Cloud Environment," in *Proceedings of International Conference on Communication and Networks*, vol. 508, N. Modi, P. Verma, and B. Trivedi, Eds. Singapore: Springer Singapore, 2017, pp. 323–331. doi: [10.1007/978-981-10-2750-5_34](https://doi.org/10.1007/978-981-10-2750-5_34).
- [9] Xiao-Bai Li and Luvai Motiwalla , "Protecting Patient Privacy with Data Masking" , WISP, 2009.
- [10] Ambrosia, Vincent G., et al. "An integration of remote sensing, GIS, and information distribution for wildfire detection and management." *Photogrammetric Engineering and Remote Sensing* 64 (1998): 977-986.
- [11] J. Kepner et al., "Computing on masked data: a high performance method for improving big data veracity," in 2014 IEEE High Performance Extreme Computing Conference (HPEC), Waltham, MA, USA, Sep. 2014, pp. 1–6. doi: [10.1109/HPEC.2014.7040946](https://doi.org/10.1109/HPEC.2014.7040946).
- [12] V. Gadepally et al., "Computing on Masked Data to improve the security of big data," in 2015 IEEE International Symposium on Technologies for Homeland Security (HST), Waltham, MA, USA, Apr. 2015, pp. 1–6. doi: [10.1109/THS.2015.7225312](https://doi.org/10.1109/THS.2015.7225312).

- [13] Acquisti, A., & Gross, R. (2009). Predicting Social Security numbers from public data. *Proceedings of the National Academy of Sciences of the United States of America*, 106(27), 10975–10980. <https://doi.org/10.1073/pnas.0904891106>
- [14] G. Sarada, N. Abitha, G. Manikandan and N. Sairam, "A few new approaches for data masking," 2015 International Conference on Circuits, Power and Computing Technologies [ICCPCT-2015], 2015, pp. 1-4, doi: 10.1109/ICCPCT.2015.7159301.
- [15] Ravikumar, Manjunath, T. N., Hegadi, R. S., & Umesh, I. M. (2011). A Survey on Recent Trends, Process and Development in Data Masking for Testing. *International Journal of Computer Science Issues*, 8(2), 535–535.
- [16] Ceccato, M., Di Penta, M., Falcarin, P., Ricca, F., Torchiano, M. & Tonella, P. 2014, "A family of experiments to assess the effectiveness and efficiency of source code obfuscation techniques", *Empirical Software Engineering*, vol. 19, no. 4, pp. 1040-1074.
- [17] Noel Hendrick and Vicenç Torra. 2021. Genetic Algorithms in Data Masking: Towards Privacy as a Service. In *Artificial Intelligence and Soft Computing: 20th International Conference, ICAISC 2021, Virtual Event, June 21–23, 2021, Proceedings, Part I*. Springer-Verlag, Berlin, Heidelberg, 381–391. https://doi.org/10.1007/978-3-030-87986-0_34